

AWS Assignment

Amazon S3

Storage, Versioning & Static Website Hosting

XYZ Corporation – step-by-step implementation with screenshots

Assignment	Topic	What was done
1	Bucket and objects	Created xyz-corporation-file-storage-2026 and uploaded 5 files (.txt, .csv, .pdf, .jpg, .html); made them publicly readable
2	Versioning	Enabled bucket versioning and re-uploaded application.txt to create new versions
3	Static website and lifecycle	Hosted index.html / error.html as a static website; lifecycle rule to Standard-IA at 60 days and expiry at 200 days

Bucket: xyz-corporation-file-storage-2026 **Region:** US East (N. Virginia) – us-east-1

Contents

Assignment 1: Create an S3 Bucket and Upload Objects.....	3
Step 1: Create the S3 bucket.....	3
Step 2: Upload five files with different extensions.....	3
Step 3: Try to open an object using its URL.....	4
Step 4: Turn off Block Public Access.....	5
Step 5: Add a bucket policy for public read.....	6
Step 6: Verify the files are publicly accessible.....	7
Assignment 2: Enable Bucket Versioning.....	9
Step 1: Check the current versioning status.....	9
Step 2: Enable versioning.....	10
Step 3: Re-upload a file.....	11
Step 4: Verify the latest version is served.....	12
Step 5: Review the version history.....	13
Assignment 3: Static Website Hosting and Lifecycle Rule.....	15
Step 1: Create and upload index.html and error.html.....	15
Step 2: Enable static website hosting.....	16
Step 3: Get the website endpoint.....	17
Step 4: Test the website.....	17
Step 5: Add the lifecycle rule.....	17
Conclusion.....	19

Assignment 1: Create an S3 Bucket and Upload Objects

Problem statement: XYZ Corporation's application requires a storage service that can store files and share them publicly if required. Implement Amazon S3 for the same.

Tasks:

1. Create an S3 bucket for file storage.
2. Upload 5 objects with different file extensions.

Step 1: Create the S3 bucket

1. Open the **Amazon S3** console and click **Create bucket**.
2. Bucket type: **General purpose**. Bucket name: **xyz-corporation-file-storage-2026** (bucket names must be globally unique).
3. Region: **US East (N. Virginia) us-east-1**. Keep the other settings at their defaults (Block all public access on, versioning disabled).
4. Click **Create bucket**.

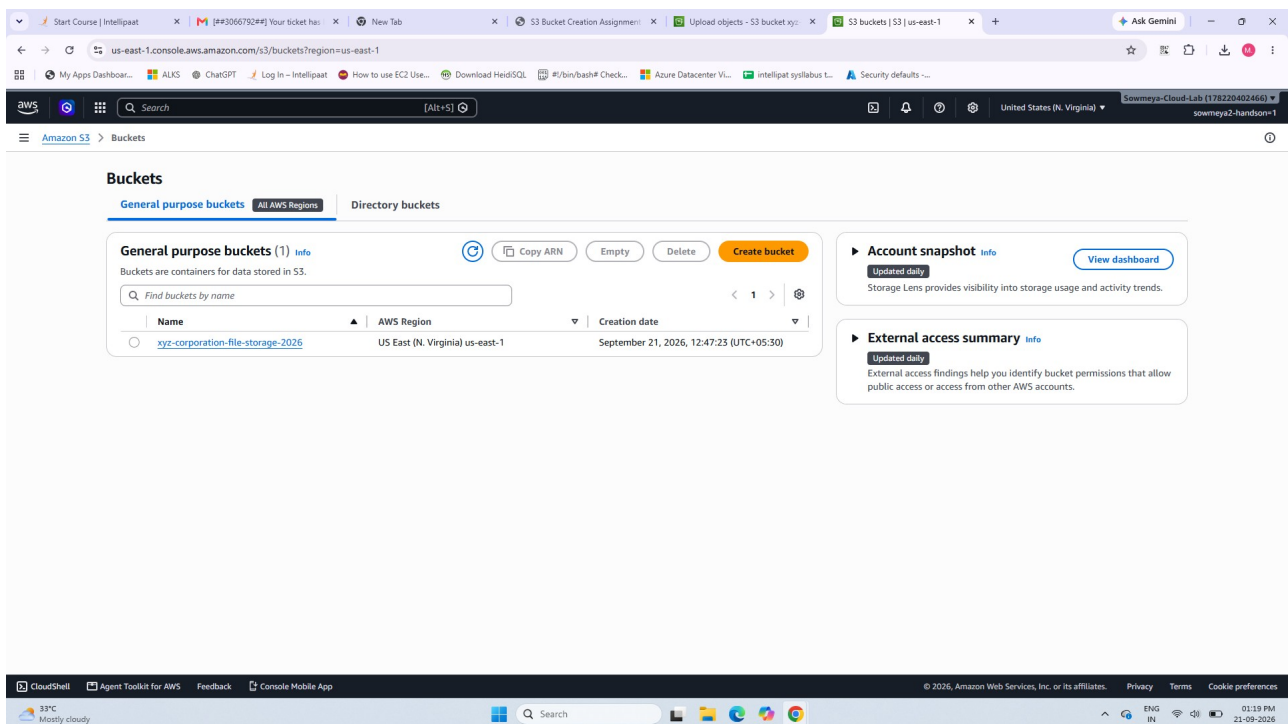


Figure 1: Bucket xyz-corporation-file-storage-2026 created in us-east-1

Step 2: Upload five files with different extensions

1. Open the bucket and click **Upload → Add files**.
2. Select the five sample files below and click Upload.

File	Extension	Size
application.txt	.txt (text)	78 B
data.csv	.csv (spreadsheet data)	73 B
document.pdf	.pdf (document)	477 B
image.jpg	.jpg (image)	170 B
website.html	.html (web page)	180 B

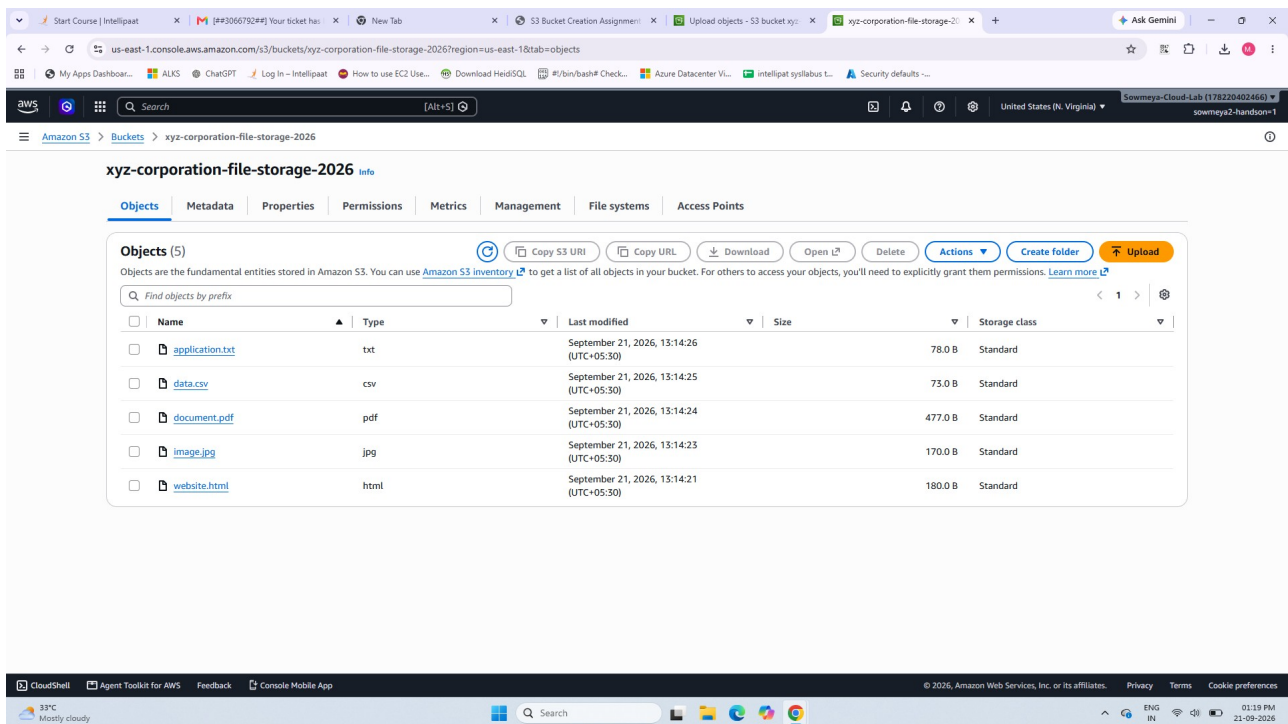


Figure 2: Five objects uploaded – txt, csv, pdf, jpg and html (Standard storage class)

Step 3: Try to open an object using its URL

1. Click **application.txt** and copy the **Object URL**.
2. Paste the URL in a new browser tab.

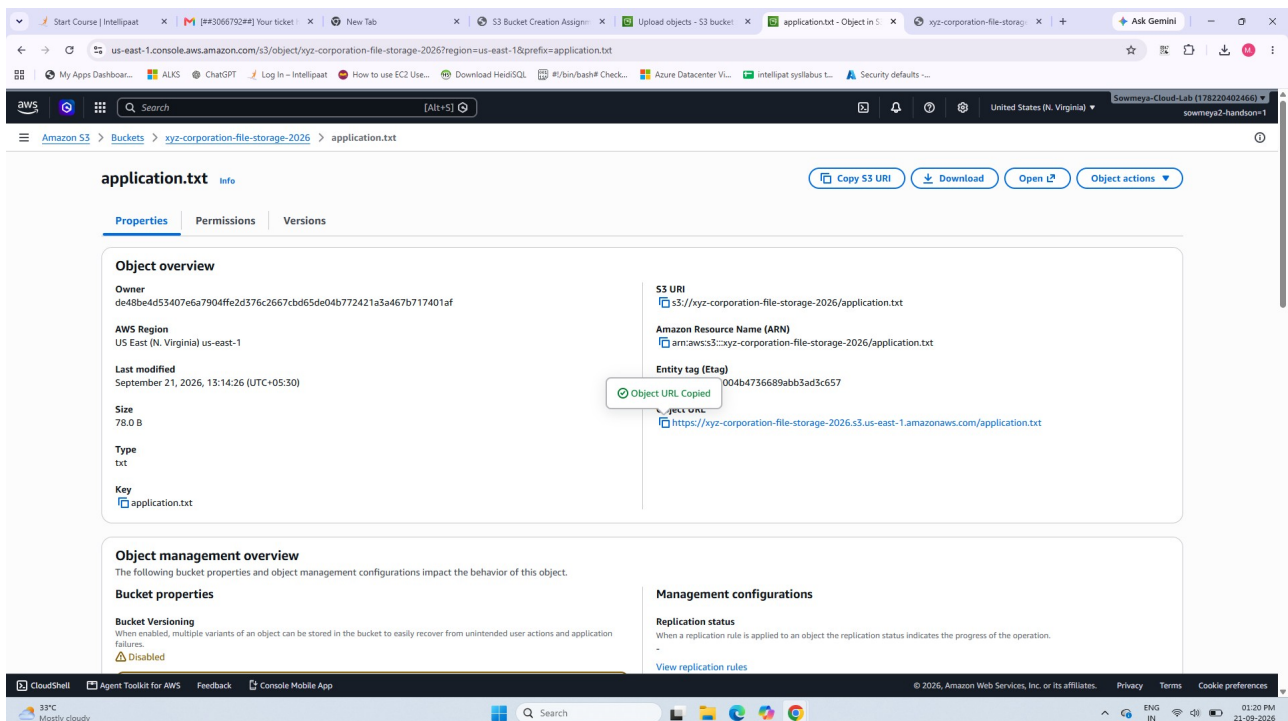


Figure 3: application.txt properties – Object URL copied

The request returns **AccessDenied** because new buckets block all public access by default. The next steps allow public read access so files can be shared.

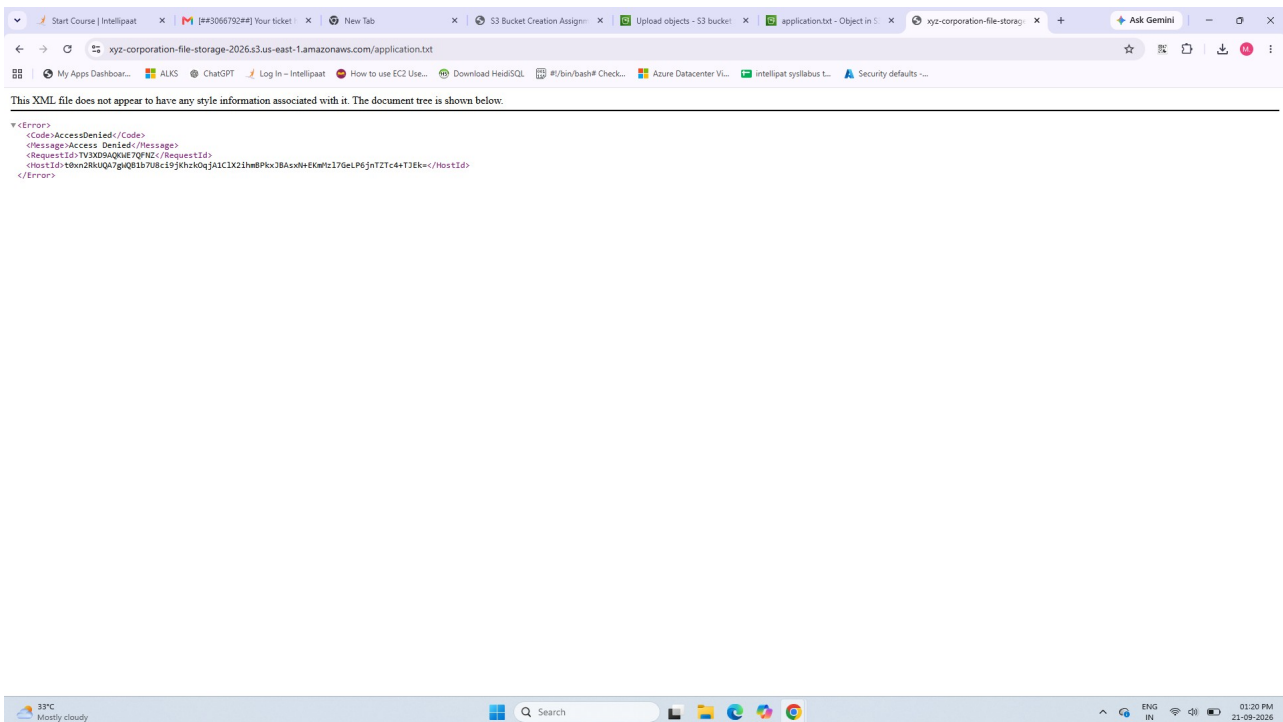


Figure 4: AccessDenied when opening the object URL before public access is allowed

Step 4: Turn off Block Public Access

1. Go to **Permissions** → **Block public access (bucket settings)** → **Edit**.
2. Untick **Block all public access**, click **Save changes**, type **confirm** and confirm.

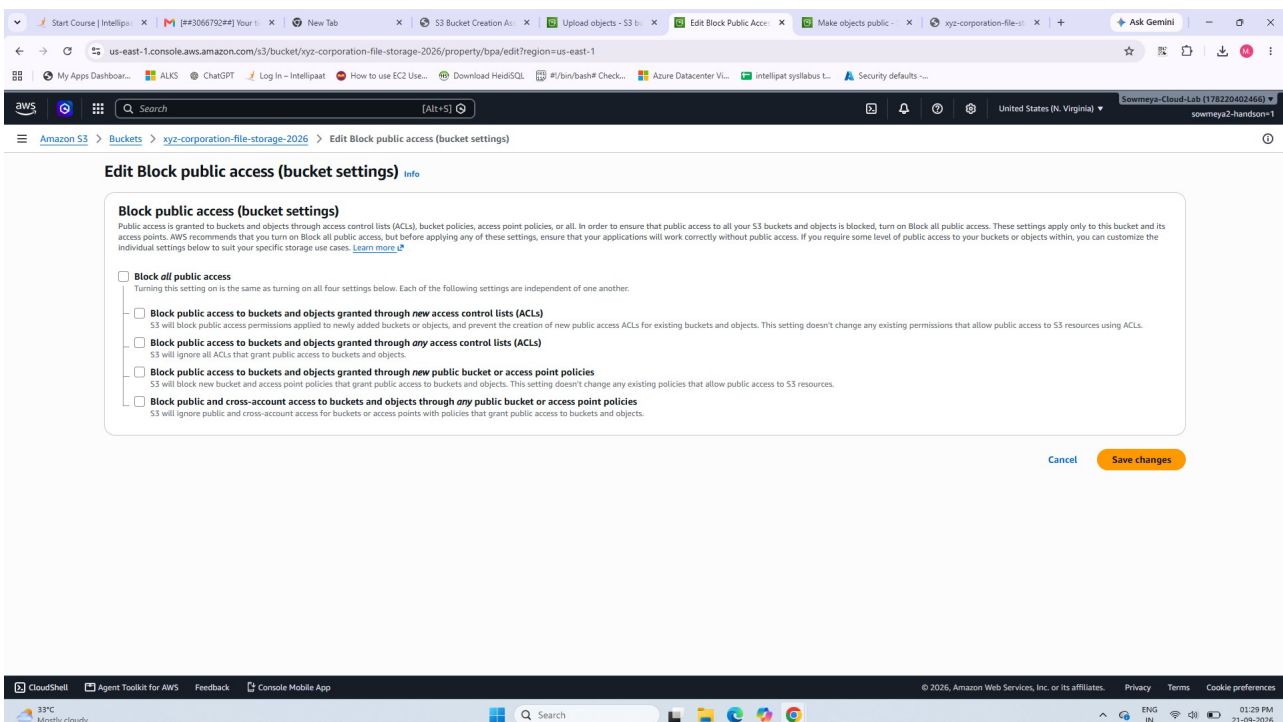


Figure 5: Edit Block public access – all four settings unticked

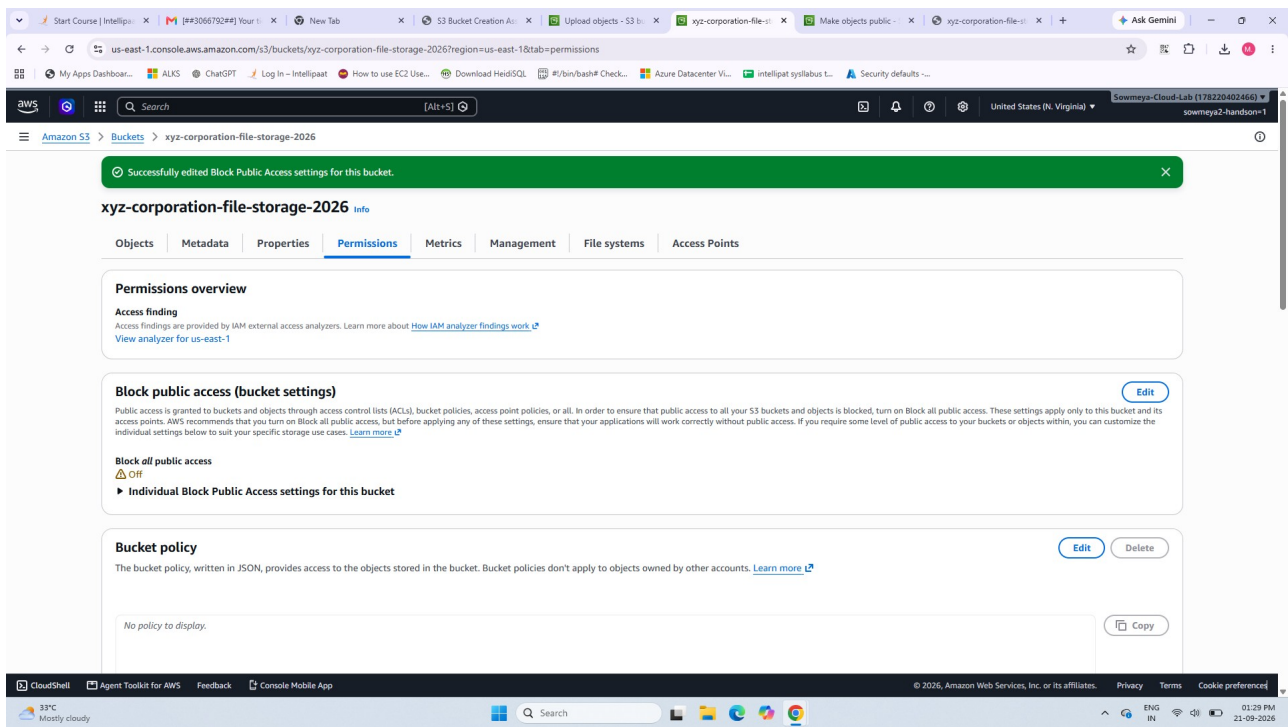


Figure 6: Block all public access is now Off; bucket policy is still empty

Step 5: Add a bucket policy for public read

1. In **Permissions** → **Bucket policy**, click **Edit**, paste the policy below and click **Save changes**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::xyz-corporation-file-storage-2026/*"
    }
  ]
}
```

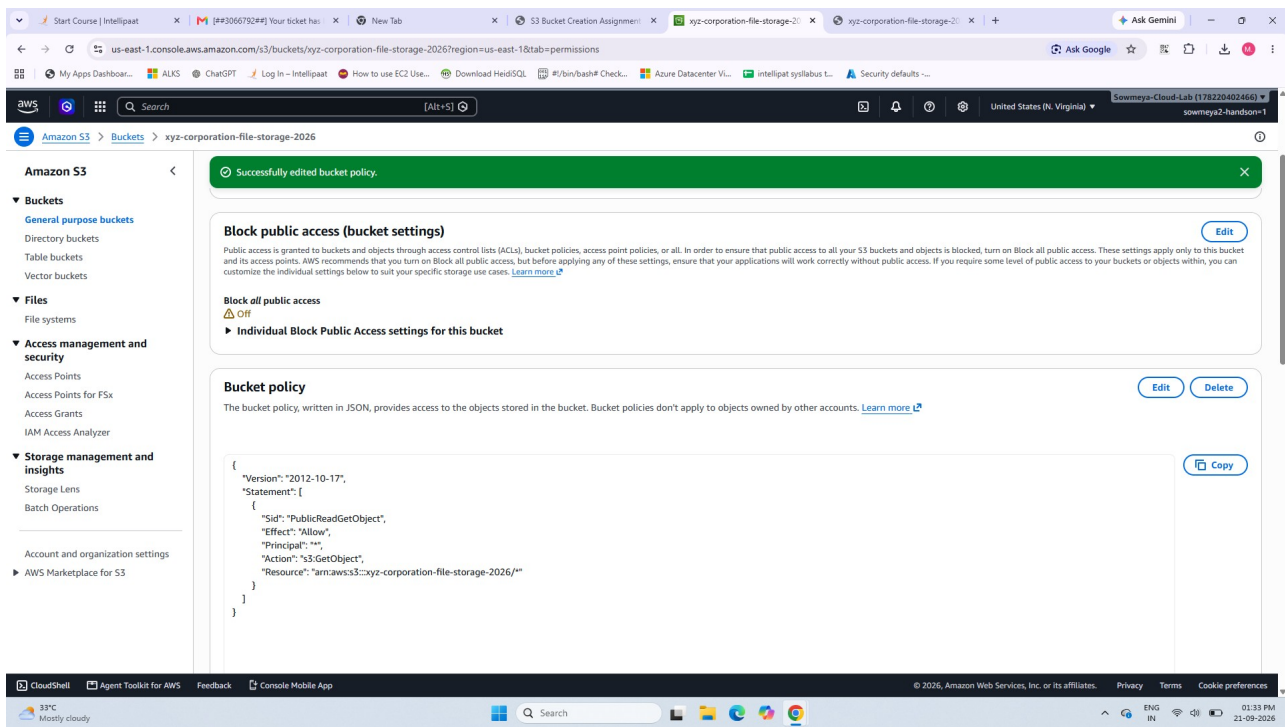


Figure 7: Bucket policy saved – public s3:GetObject on every object in the bucket

Note: This policy lets anyone on the internet read every file in the bucket. Only use it for files that are meant to be public.

Step 6: Verify the files are publicly accessible

Open the object URLs again. The files now load in the browser:

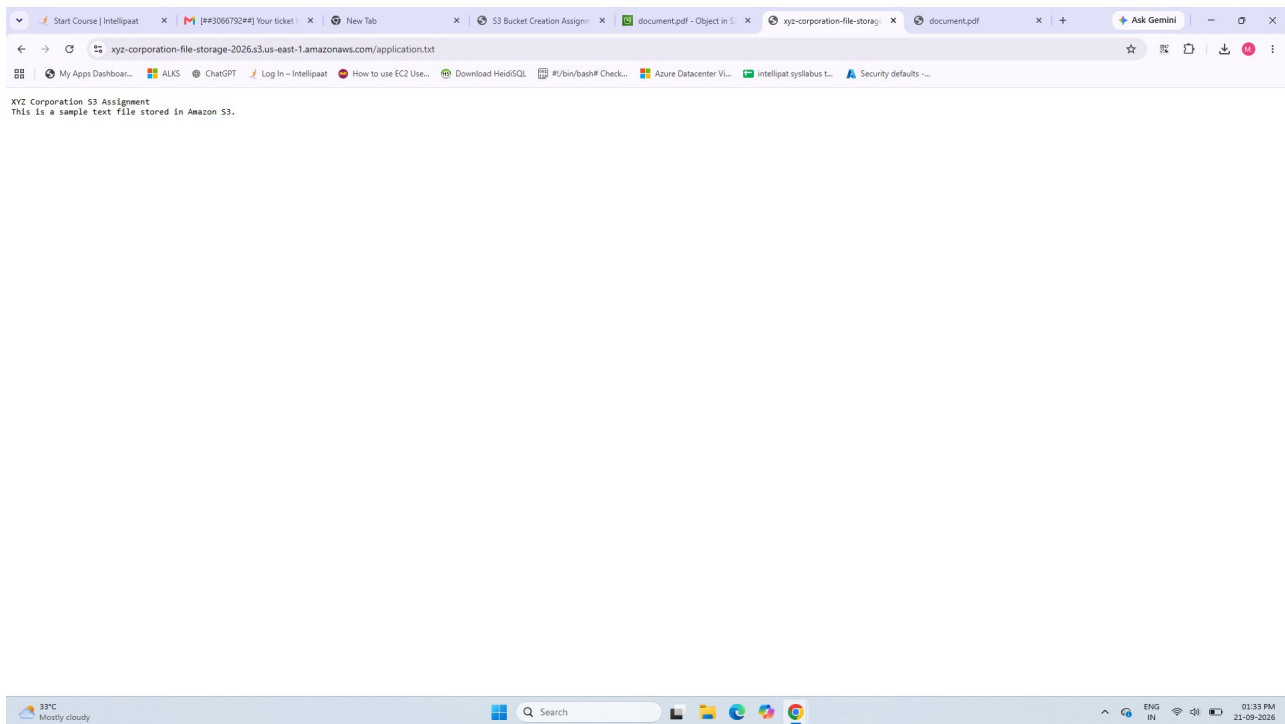


Figure 8: application.txt opened from its public object URL

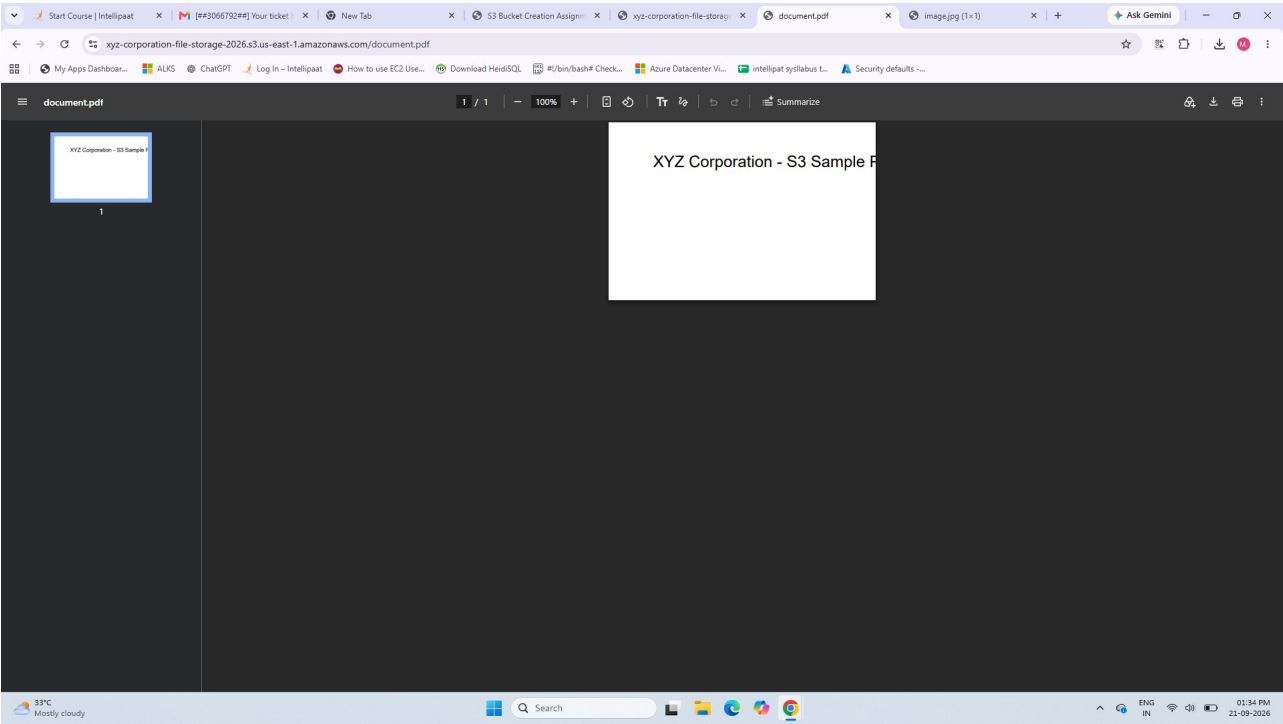


Figure 9: document.pdf opened from its public object URL

Assignment 2: Enable Bucket Versioning

Problem statement: XYZ Corporation's application requires a storage service that can store files and share them publicly if required. Implement Amazon S3 for the same.

Tasks:

1. Enable versioning for the bucket created in Assignment 1.
2. Re-upload any 2 files that were already uploaded to verify that versioning works.

Step 1: Check the current versioning status

The bucket still holds the five files from Assignment 1. On the **Properties** tab, **Bucket Versioning** shows **Disabled**.

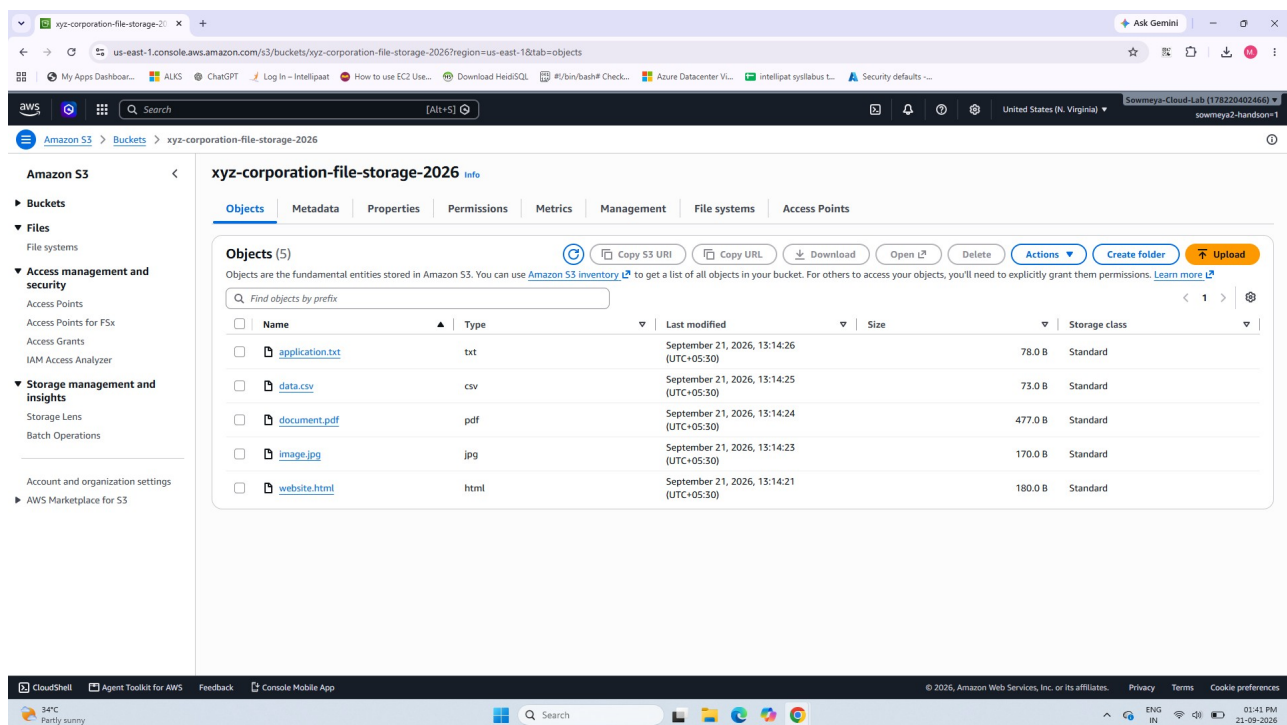


Figure 10: Bucket objects before versioning is enabled

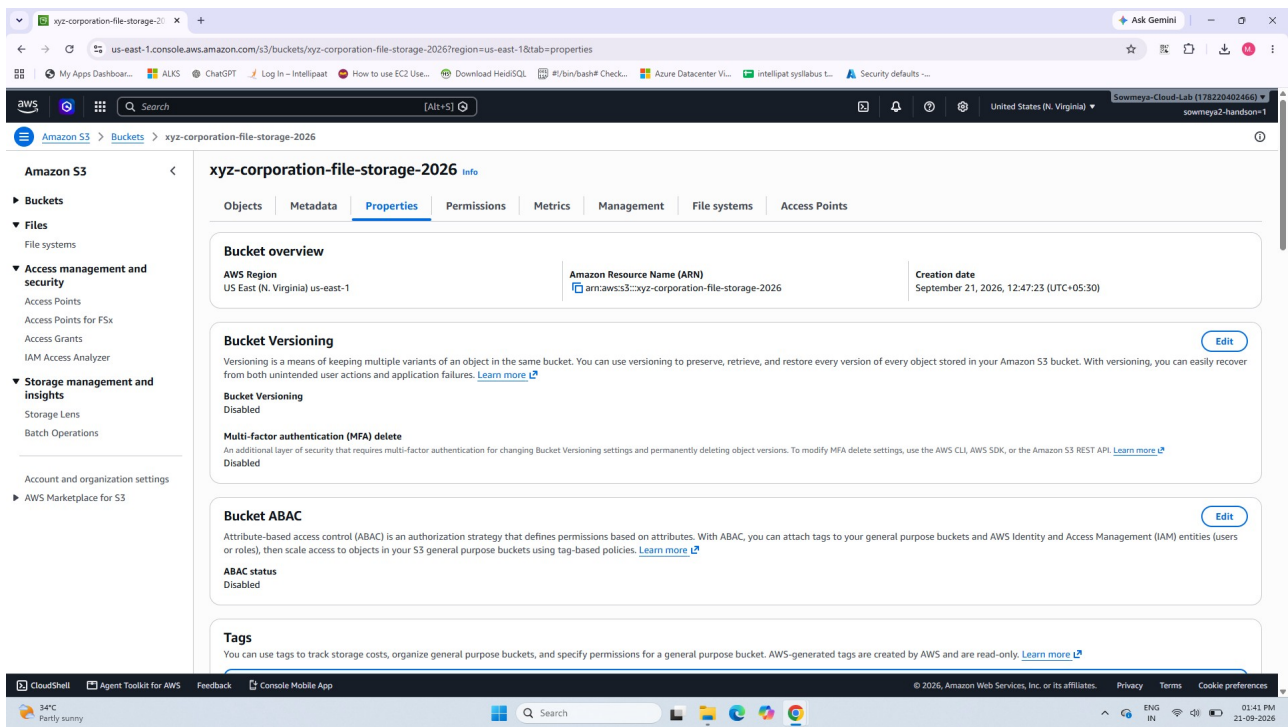


Figure 11: Properties – Bucket Versioning Disabled

Step 2: Enable versioning

1. In **Properties** → **Bucket Versioning**, click **Edit**.
2. Select **Enable** and click **Save changes**.

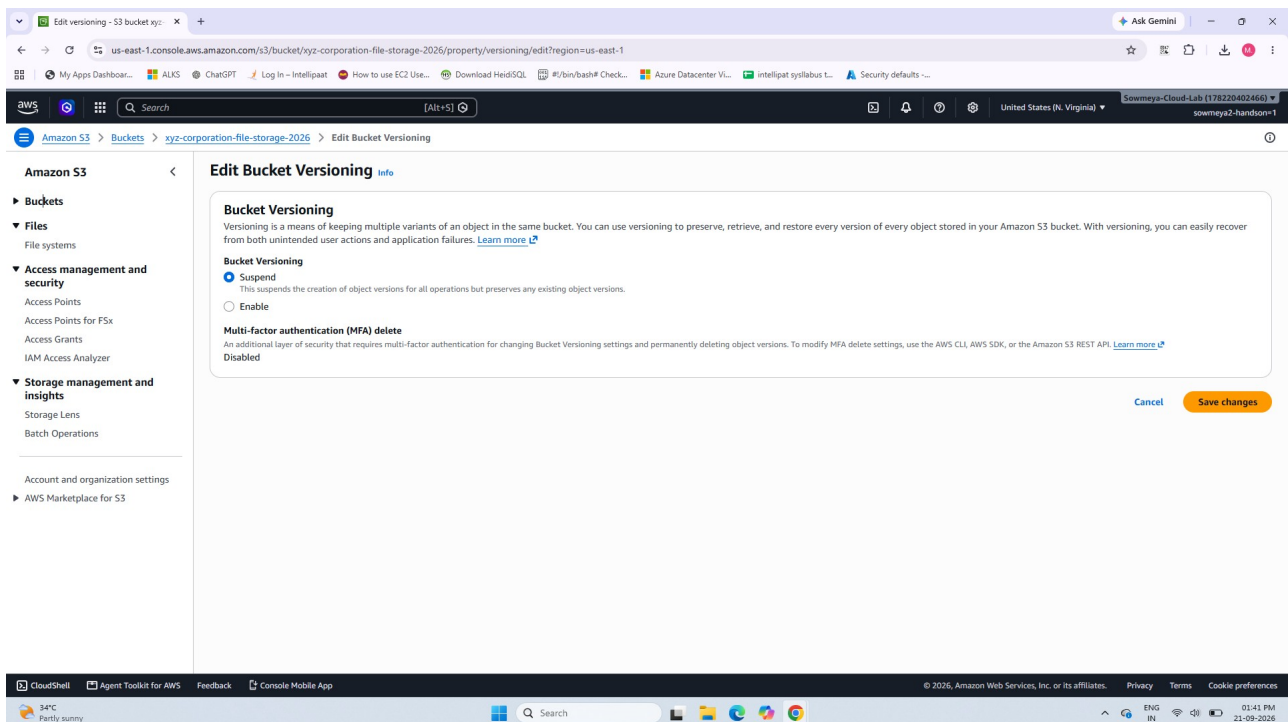


Figure 12: Edit Bucket Versioning page

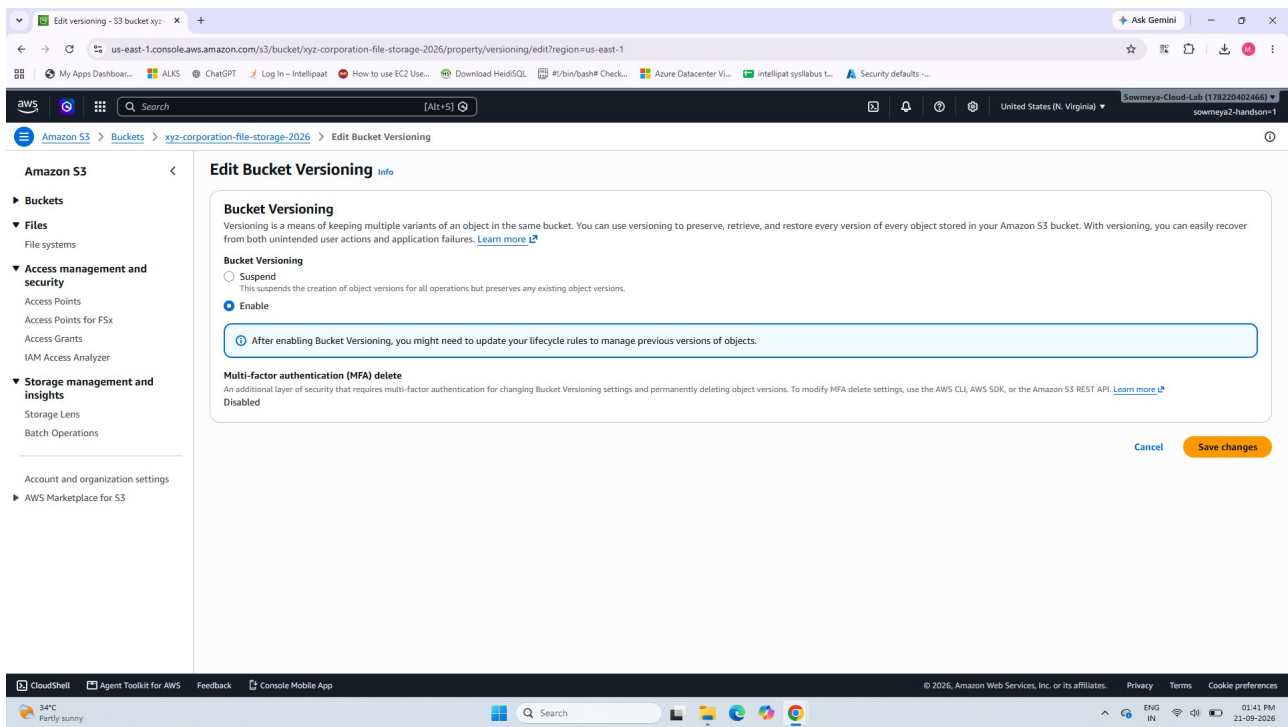


Figure 13: Enable selected – click Save changes

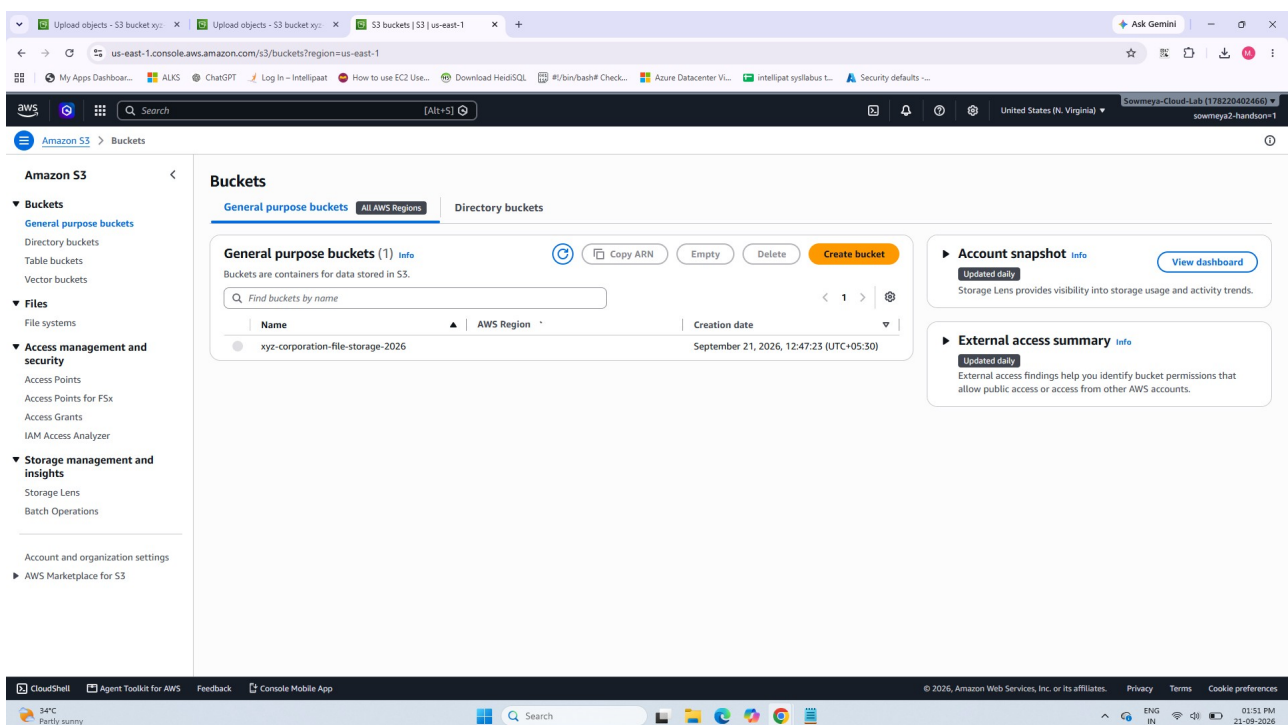


Figure 14: Buckets list after saving the versioning change

Good to know: Once versioning has been enabled it can only be suspended, not turned off. Files uploaded before versioning keep the version ID “null”.

Step 3: Re-upload a file

1. Edit **application.txt** on your computer (for example add the line **Versioning -1**).
2. In the bucket click **Upload → Add files**, choose the edited file with the same name and click **Upload**.
3. Turn on **Show versions** in the Objects tab to see every version.

Objects (7)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix: Show versions: ☒

☐	Name	Type	Version ID	Last modified	Size	Storage class
☐	application.txt	txt	ON8VhVSI5VDvB0mxiqQqt.ngkrw4e	September 21, 2026, 13:50:29 (UTC+05:30)	93.0 B	Standard
☐	application.txt	txt	xv_PNPdZTV0Hw2zrRGZD2dPyEQJjSyGv	September 21, 2026, 13:49:26 (UTC+05:30)	93.0 B	Standard
☐	application.txt	txt	null	September 21, 2026, 13:14:26 (UTC+05:30)	78.0 B	Standard
☐	data.csv	csv	null	September 21, 2026, 13:14:25 (UTC+05:30)	73.0 B	Standard
☐	document.pdf	pdf	null	September 21, 2026, 13:14:24 (UTC+05:30)	477.0 B	Standard
☐	image.jpg	jpg	null	September 21, 2026, 13:14:23 (UTC+05:30)	170.0 B	Standard
☐	website.html	html	null	September 21, 2026, 13:14:21 (UTC+05:30)	180.0 B	Standard

Figure 15: Show versions – application.txt now has new versions above the original null version

Upload: status

After you navigate away from this page, the following information is no longer available.

Summary

Destination: [s3://xyz-corporation-file-storage-2026](#)

Succeeded: ☒ 1 file, 93.0 B (100.00%)

Failed: ☐ 0 files, 0 B (0%)

Files and folders

Files and folders (1 total, 93.0 B)

Find by name:

Name	Folder	Type	Size	Status	Error
application.txt	-	text/plain	93.0 B	☒ Succeeded	-

Figure 16: Upload succeeded – application.txt (93 B)

Step 4: Verify the latest version is served

Open the object URL. S3 always returns the newest version. After each re-upload the content changes from **Versioning -1** to **Versioning -2**.

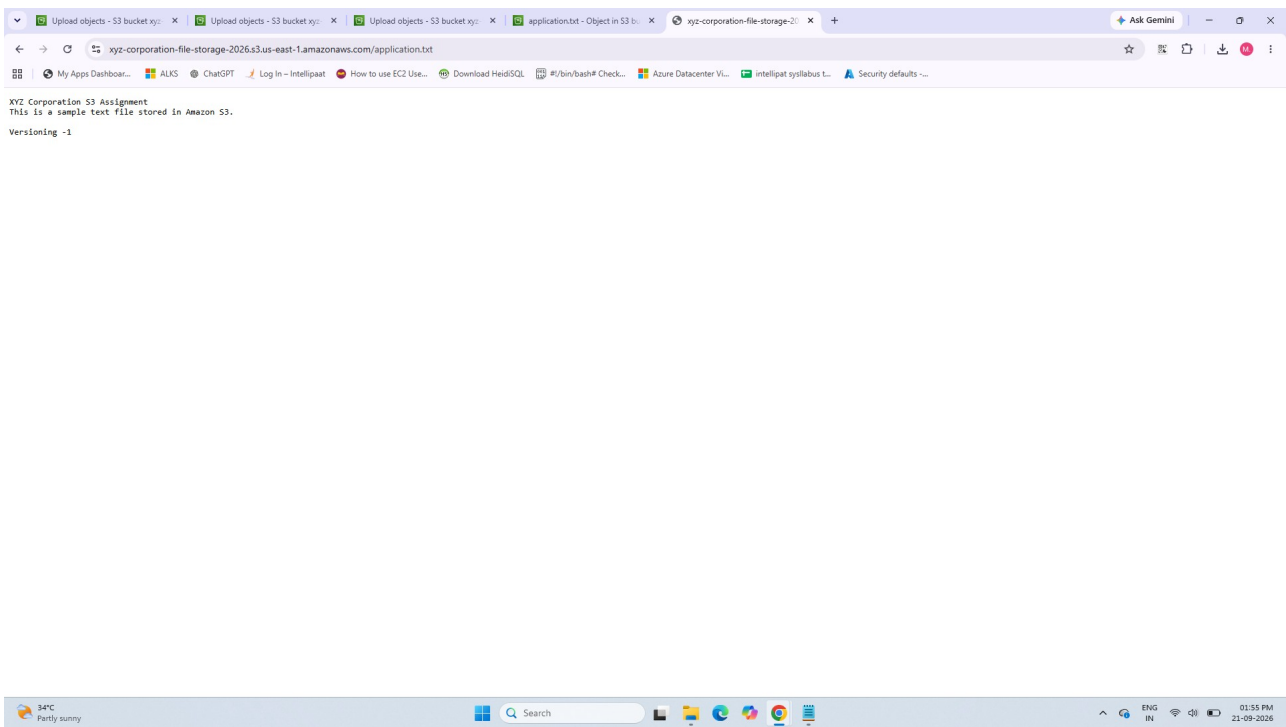


Figure 17: Object URL returns the content of the first re-upload (Versioning -1)

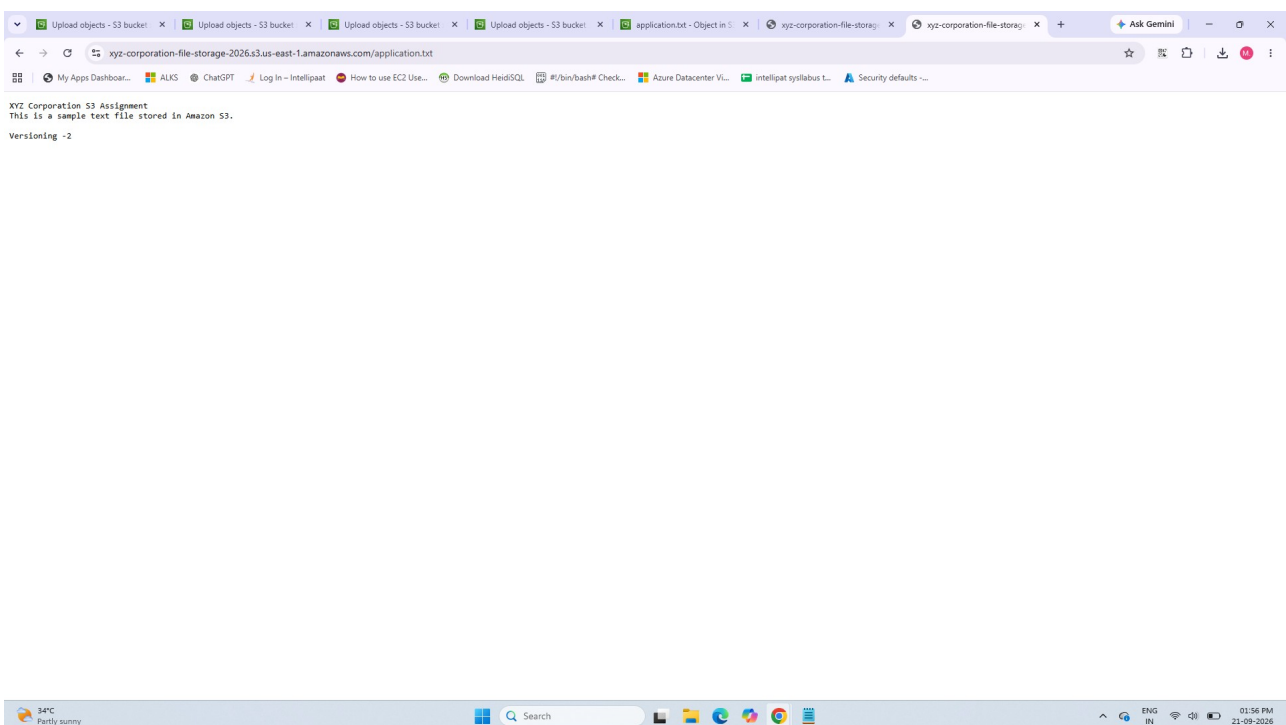


Figure 18: Object URL returns the content of the second re-upload (Versioning -2)

Step 5: Review the version history

Each upload kept its own version ID, so any earlier version can be downloaded or restored from the Versions list:

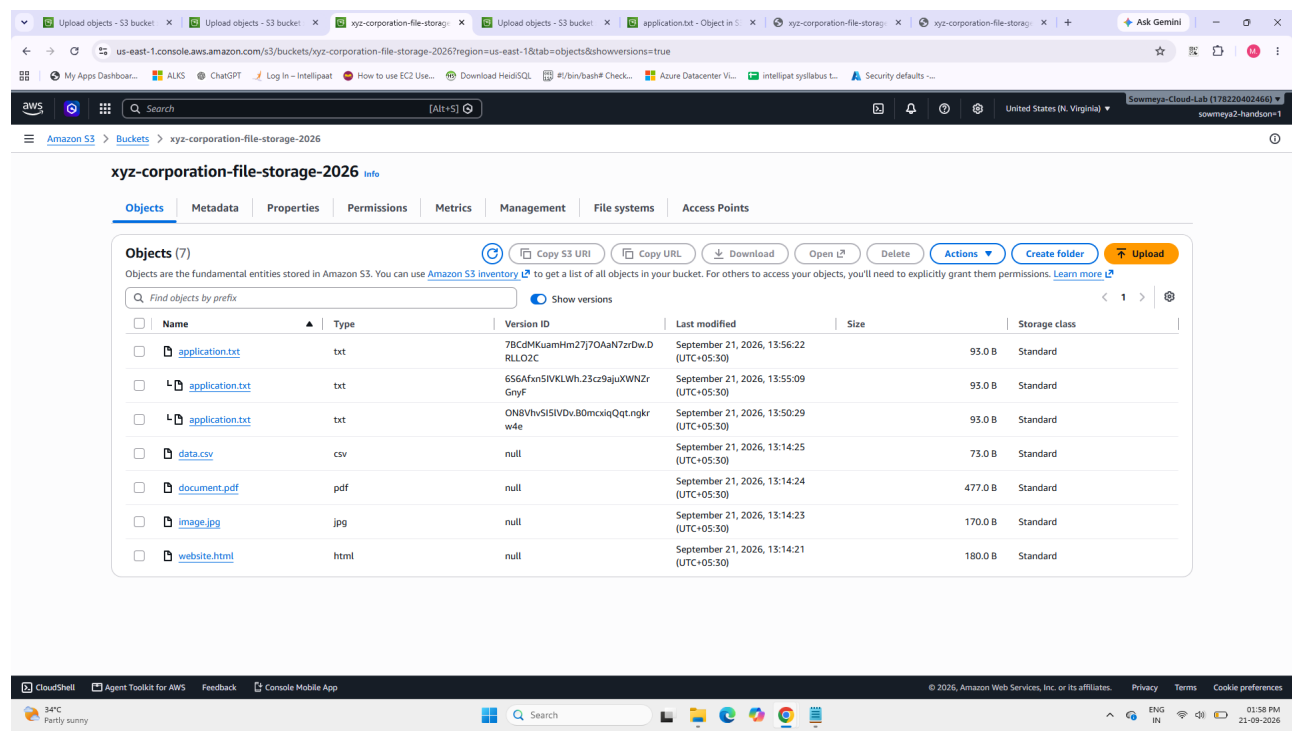


Figure 19: Version history – three versions of application.txt with different version IDs

Assignment 3: Static Website Hosting and Lifecycle Rule

Problem statement: XYZ Corporation's application requires a storage service that can store files and share them publicly if required. Implement Amazon S3 for the same.

Tasks:

1. Use the bucket from the previous task to host a static website; upload an index.html file and an error.html page.
2. Add a lifecycle rule for the bucket: (a) transition from Standard to Standard-IA after 60 days, (b) expire objects after 200 days.

Step 1: Create and upload index.html and error.html

Example **index.html**:

```
<!DOCTYPE html>
<html>
<head><title>XYZ Corporation</title></head>
<body>
  <h1>Welcome to XYZ Corporation</h1>
  <p>This is our static website hosted on Amazon S3.</p>
</body>
</html>
```

Example **error.html**:

```
<!DOCTYPE html>
<html>
<head><title>Error - XYZ Corporation</title></head>
<body>
  <h1>404 - Page Not Found</h1>
  <p>Sorry, the page you requested does not exist.</p>
</body>
</html>
```

1. In the bucket click **Upload → Add files**, select **index.html** and **error.html**, then **Upload**.

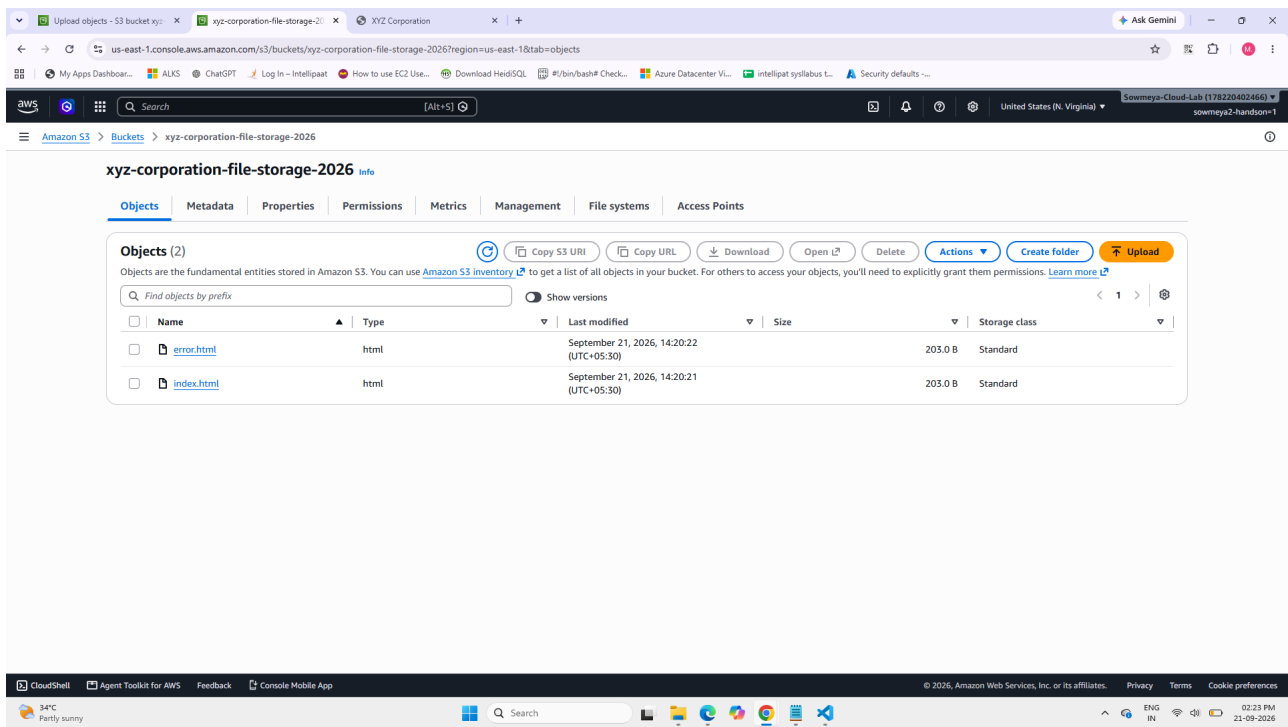


Figure 20: index.html and error.html uploaded to the bucket

Step 2: Enable static website hosting

1. Go to **Properties** → **Static website hosting** → **Edit**.
2. Select **Enable** and hosting type **Host a static website**.
3. Index document: **index.html**. Error document: **error.html**. Click **Save changes**.

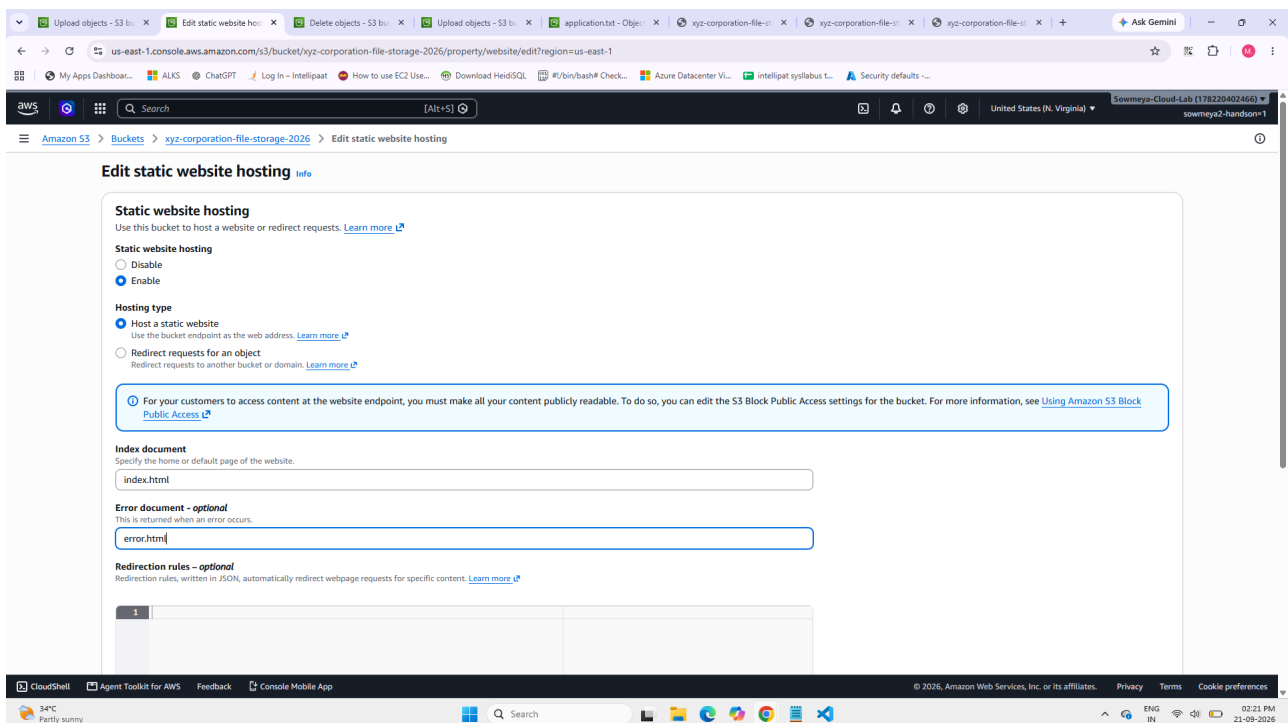


Figure 21: Edit static website hosting – index.html and error.html

Public access: The website only works if visitors can read the files. Block Public Access is already off and the public-read bucket policy from Assignment 1 (Step 5) already covers index.html and error.html.

Step 3: Get the website endpoint

Scroll to the bottom of the **Properties** tab. Static website hosting shows **Enabled** and the bucket website endpoint:

`http://xyz-corporation-file-storage-2026.s3-website-us-east-1.amazonaws.com`

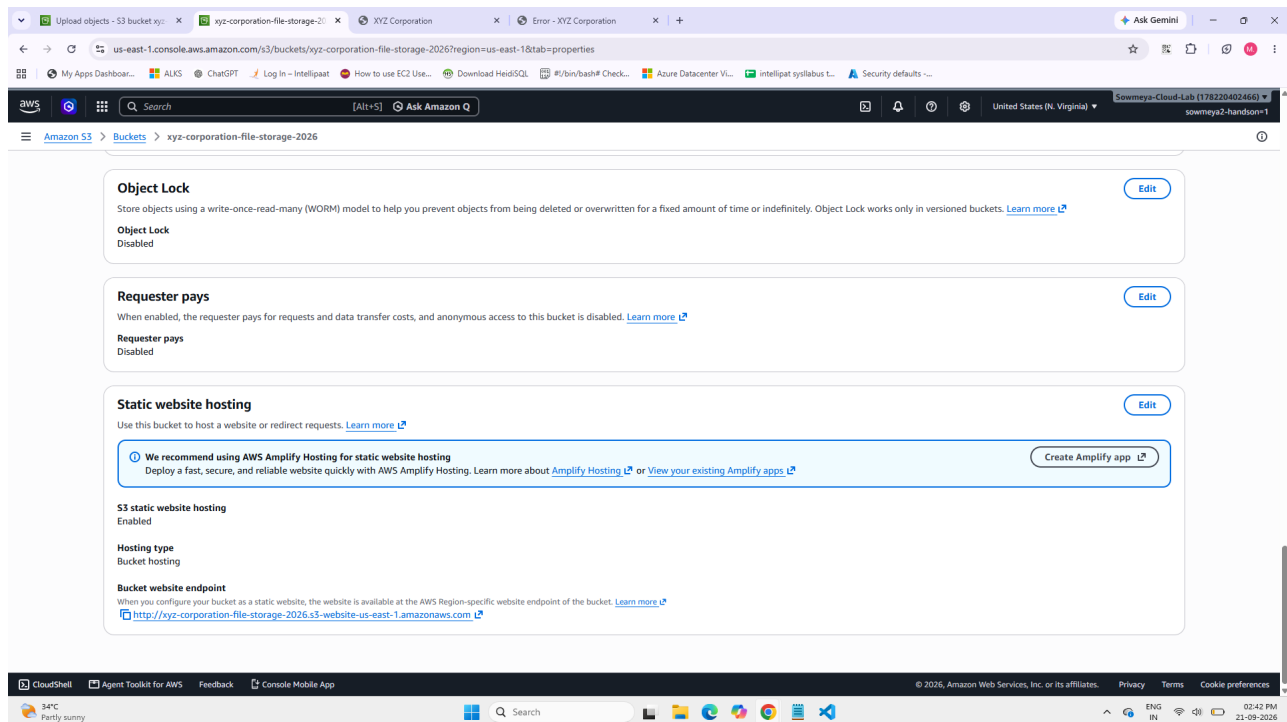


Figure 22: Static website hosting Enabled with the bucket website endpoint

Step 4: Test the website

1. Open the website endpoint in a browser – the index.html page is displayed.
2. Open a page that does not exist (for example `/missing.html`) – S3 returns the error.html page.

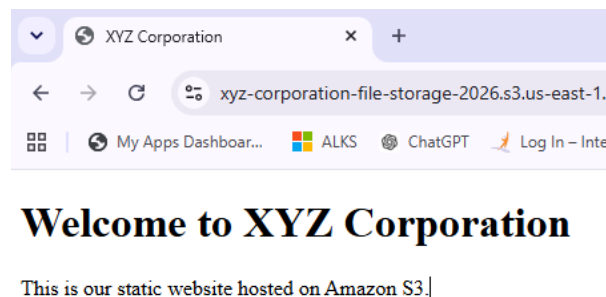


Figure 23: Welcome to XYZ Corporation – index.html served from S3

Step 5: Add the lifecycle rule

1. Go to **Management** → **Lifecycle rules** → **Create lifecycle rule**.

2. Rule name: **XYZ-Lifecycle-Rule**. Scope: **Apply to all objects in the bucket** and tick the acknowledgement.
3. Rule actions: tick **Transition current versions of objects between storage classes** and **Expire current versions of objects**.
4. Transition: **Standard-IA**, **60** days after object creation.
5. Expiration: **200** days after object creation. Click **Create rule**.

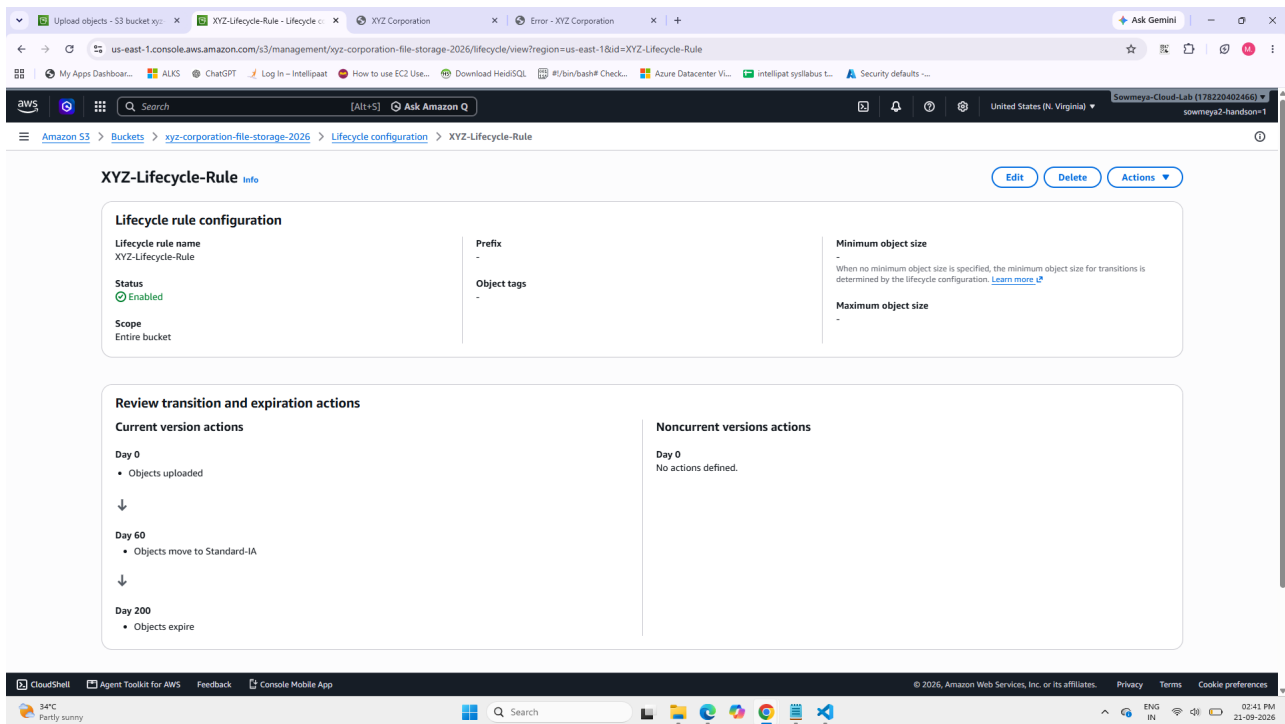


Figure 24: XYZ-Lifecycle-Rule – Day 60 move to Standard-IA, Day 200 objects expire

Versioned bucket: Because versioning is on, expiring a current version adds a delete marker and keeps the old version as noncurrent. To remove old versions too, add a “Permanently delete noncurrent versions of objects” action to the same rule.

Conclusion

Each task maps to the configuration and evidence below:

Assignment / task	How it was met	Evidence
1.1 Create S3 bucket	xyz-corporation-file-storage-2026 in us-east-1	Figure 1
1.2 Upload 5 objects	application.txt, data.csv, document.pdf, image.jpg, website.html	Figure 2
Public sharing	Block Public Access off and public-read bucket policy	Figures 3–9
2.1 Enable versioning	Bucket Versioning set to Enabled	Figures 11–13
2.2 Re-upload files	application.txt re-uploaded; new version IDs created and latest content served	Figures 15–19
3.1 Static website	index.html and error.html uploaded, website hosting enabled	Figures 20–23
3.2a Standard-IA at 60 days	XYZ-Lifecycle-Rule transition action	Figure 24
3.2b Expire at 200 days	XYZ-Lifecycle-Rule expiration action	Figure 24